

STRAVA FITNESS

Python Report (Data Visualization)

This document details the second phase of the Strava Fitness analysis project: exploratory data analysis (EDA) and visualization using Python. This report is a direct continuation of the SQL Data Loading & Cleaning Report. The primary objective of this phase was to connect to the cleaned strava MySQL database, query the data, and generate a series of visualizations to answer key business questions about user behavior. The main Python libraries used were: SQLAlchemy, mysql-connector-python, Pandas, Matplotlib and Seaborn

- **DataBase Connection :**

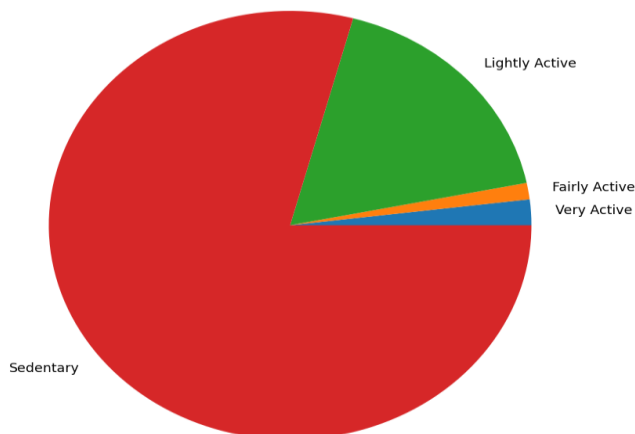
A reusable database engine was created using SQLAlchemy to manage the connection. This engine was used for all subsequent queries.

```
import pandas as pd
from sqlalchemy import create_engine
import matplotlib.pyplot as plt
import seaborn as sns
import calendar
import sys
DB_PASSWORD='Sumit_123'
db_connection_str=f'mysql+mysqlconnector://root:{DB_PASSWORD}@127.0.0.1:3306/strava'
db_engine=create_engine(db_connection_str)
print("Database connection successful.")
```

- **Daily Activity Consumption :**

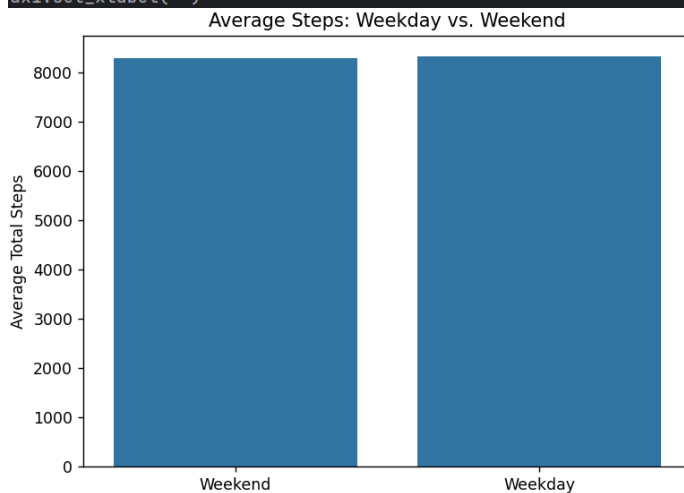
```
activity_query="""
    SELECT SUM(VeryActiveMinutes) AS "Very Active", SUM(FairlyActiveMinutes) AS "Fairly Active",
           SUM(LightlyActiveMinutes) AS "Lightly Active", SUM(SedentaryMinutes) AS "Sedentary"
    FROM dailyactivity WHERE SedentaryMinutes < 1440 AND TotalSteps > 0"""
df_activity_totals=pd.read_sql(activity_query, db_engine)
activity_data=df_activity_totals.iloc[0]
activity_data=activity_data.reset_index()
activity_data.columns=['ActivityType', 'TotalMinutes']
plt.figure(figsize=(8, 8))
plt.pie(activity_data['TotalMinutes'], labels=activity_data['ActivityType'])
plt.title(label: 'How Users Spend Their Day (Based on Total Minutes)', fontsize=16)
```

How Users Spend Their Day (Based on Total Minutes)



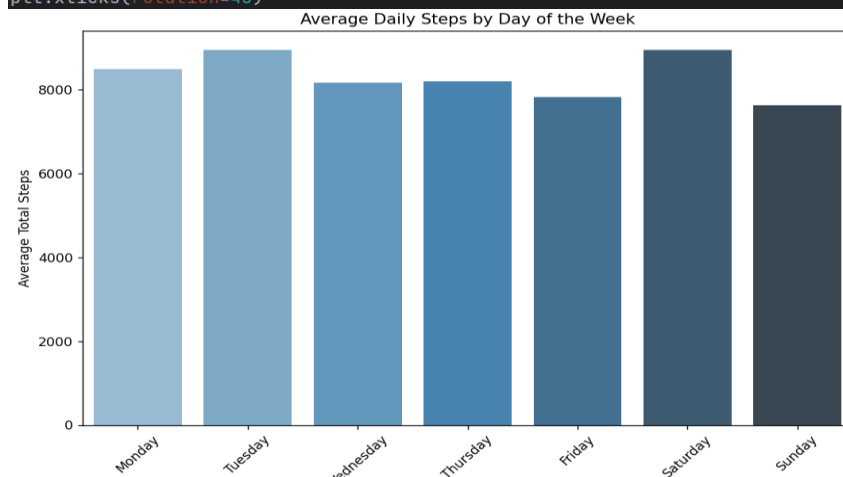
- **WeekDay vs WeekEnd Activity :**

```
activity_df=pd.read_sql(sql: "SELECT ActivityDate, TotalSteps FROM dailyactivity WHERE TotalSteps > 0", db_engine)
activity_df['ActivityDate']=pd.to_datetime(activity_df['ActivityDate'])
activity_df['DayOfWeek']=activity_df['ActivityDate'].dt.day_name()
activity_df['WeekPart']=activity_df['DayOfWeek'].apply(
    lambda x: 'Weekend' if x in ['Saturday', 'Sunday'] else 'Weekday')
avg_steps_weekpart=activity_df.groupby('WeekPart')['TotalSteps'].mean().reset_index()
avg_steps_weekpart=avg_steps_weekpart.sort_values(by='WeekPart', ascending=False)
plt.figure(figsize=(7, 5))
ax1=sns.barplot(data=avg_steps_weekpart, x='WeekPart', y='TotalSteps',)
ax1.set_title('Average Steps: Weekday vs. Weekend')
ax1.set_ylabel('Average Total Steps')
ax1.set_xlabel('')
```



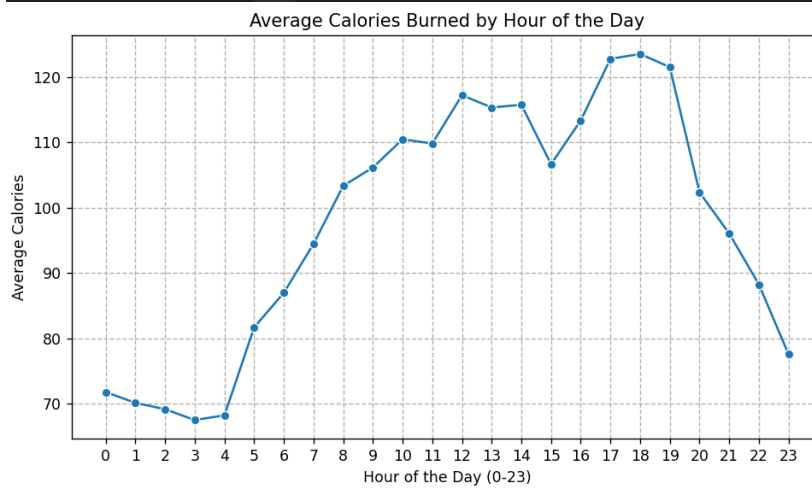
- **Total Steps per Day :**

```
avg_steps_day=activity_df.groupby('DayOfWeek')['TotalSteps'].mean().reset_index()
day_order=list(calendar.day_name)
avg_steps_day['DayOfWeek']=pd.Categorical(avg_steps_day['DayOfWeek'], categories=day_order, ordered=True)
avg_steps_day=avg_steps_day.sort_values('DayOfWeek')
plt.figure(figsize=(10, 6))
ax2=sns.barplot(
    data=avg_steps_day,
    x='DayOfWeek',
    y='TotalSteps',
    palette='Blues_d'
)
ax2.set_title('Average Daily Steps by Day of the Week')
ax2.set_ylabel('Average Total Steps')
ax2.set_xlabel('Day of the Week')
plt.xticks(rotation=45)
```



- **Calories Burn by Hour :**

```
hourly_calories_df=pd.read_sql( sql: "SELECT ActivityHour, Calories FROM hourlycalories", db_engine)
hourly_calories_df['ActivityHour']=pd.to_datetime(hourly_calories_df['ActivityHour'])
hourly_calories_df['Hour']=hourly_calories_df['ActivityHour'].dt.hour
avg_calories_hour=hourly_calories_df.groupby('Hour')['Calories'].mean().reset_index()
plt.figure(figsize=(12, 6))
ax=sns.lineplot(data=avg_calories_hour, x='Hour', y='Calories', marker='o')
ax.set_title('Average Calories Burned by Hour of the Day')
ax.set_ylabel('Average Calories')
ax.set_xlabel('Hour of the Day (0-23)')
ax.set_xticks(range(0, 24)) # Set ticks for every hour
plt.grid( visible: True, which='both', linestyle='--')
```



- **Sleep Efficiency :**

```
sleep_df=pd.read_sql( sql: "SELECT TotalTimeInBed, TotalMinutesAsleep FROM sleepday_clean WHERE TotalMinutesAsleep > 0")
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=sleep_df,
    x='TotalTimeInBed',
    y='TotalMinutesAsleep',
    alpha=0.6
)
sns.regplot(
    data=sleep_df,
    x='TotalTimeInBed',
    y='TotalMinutesAsleep',
    scatter=False,
    color='red'
)
plt.title('Time in Bed vs. Actual Time Asleep')
plt.xlabel('Total Time in Bed (Minutes)')
plt.ylabel('Total Minutes Asleep')
plt.show()
```

